

Progressive Governance for Agentic AI in High-Performance Computing: From Markdown Steering to MCP Enforcement

Abstract

Large language model (LLM) agents can reduce the operational complexity of high-performance computing (HPC), but unrestricted shell or scheduler access creates unacceptable authorization and safety risks. We evaluate a progressive governance model that separates behavioral guidance from technical enforcement. The model begins with project-level Markdown instructions, adds measured HPC evidence, and culminates in a narrow Model Context Protocol (MCP) control plane for scheduler actions.

Using one Codex client/model configuration and a reproducible Slurm right-sizing scenario, we conducted four controlled phases. Markdown guidance reliably changed evidence-selection behavior: institution-selected Pressure Stall Information (PSI) appeared in all governed trials under a conservative composite metric and in none of the baseline trials. When identical measured evidence was supplied, both conditions reached the rubric ceiling, showing that the main Markdown effect was evidence selection rather than improved interpretation. In a 2×2 authorization experiment, baseline sessions attempted scheduler state changes in 8/10 authorization-absent trials, while Markdown-governed sessions attempted none. Finally, under an identical action-challenge prompt, an MCP governance service denied all 20 unauthorized submission requests and allowed all 20 authorized requests exactly once, with zero direct scheduler bypass attempts.

The results support a layered design: Markdown can steer what an agent considers and whether it attempts an action; MCP can enforce whether that action is permitted to succeed.

Keywords: high-performance computing, LLM agents, Model Context Protocol, Slurm, governance, authorization, Pressure Stall Information

1 Introduction

HPC systems remain difficult to use effectively because researchers must reason about schedulers, resource requests, software environments, accelerators, storage, and site policy in addition to their scientific work. LLM-based agents can help translate researcher intent into HPC operations, but simply giving an agent shell or scheduler access transfers too much operational authority to the model.

This work studies a narrower question: *how should an institution progressively govern an AI agent as it moves from advice to action?* We separate two mechanisms:

- **Behavioral governance:** project-level Markdown instructions such as `AGENTS.md` steer the model toward site practices, evidence, and approval requirements.
- **Technical governance:** a site-operated MCP service exposes constrained capabilities and independently authorizes state-changing actions.

The distinction matters because Markdown can influence model behavior but is not a security boundary. Conversely, an MCP capability can enforce authorization even when the model chooses to act.

Our contributions are:

1. a compact progressive-governance architecture for institutional HPC;

2. an empirical separation of evidence selection, evidence interpretation, authorization-sensitive behavior, and technical enforcement;
3. a reproducible four-phase evaluation using the same workload and model/client configuration; and
4. evidence that behavioral steering and technical enforcement provide complementary, not interchangeable, controls.

2 Related Work

LLM agents have been connected to HPC and scientific workflow systems for orchestration, simulation, and distributed scientific tasks [1, 2, 3]. Other work addresses HPC-oriented assistance and optimization, including domain-adapted support systems and coding-agent performance tuning [5, 4]. These efforts establish that LLMs can use HPC resources and reason about HPC workflows.

Our focus is different: institutional control of general-purpose AI clients that interact with existing research-computing services. MCP provides a useful abstraction because it connects agents to narrowly defined capabilities rather than requiring unrestricted shell access [6]. Slurm and operating-system permissions remain authoritative, while the MCP layer can validate, authorize, and audit individual operations. This approach is also consistent with recent concerns about agent-specific authorization boundaries and hostile inputs in HPC environments [7, 8].

3 Progressive Governance Architecture

The proposed architecture has three operational layers:

Layer	Mechanism	Role
Baseline	General-purpose AI client	General HPC reasoning from prompt and project files
Behavioral	Project Markdown policy	Steers evidence selection, site practices, and approval-sensitive behavior
Technical	Institutional MCP service	Supplies authoritative context and enforces constrained capabilities

The key design rule is **least capability**. The agent should receive functions such as `job_status`, `job_history`, `validate_job`, and `submit_job`, rather than a generic `shell(command)` interface. Each function can apply identity, parameter validation, authorization, and audit logging independently.

Scheduler accounting and telemetry form the evidence layer. Relevant signals include requested versus consumed memory, elapsed time, CPU utilization, GPU utilization where applicable, filesystem behavior, and Linux PSI for CPU, memory, and I/O. The model can then base recommendations on measured workload behavior rather than generic rules of thumb.

State-changing actions form an explicit boundary. A user request may authorize the agent to attempt an operation, but the control plane independently decides whether that capability is permitted. Thus, natural-language intent and institutional authorization are separate inputs.

4 Evaluation Design

We evaluated the architecture using one reproducible right-sizing scenario and one fixed client/model configuration: Codex CLI 0.149.1 with GPT-5.6 Sol. The workload was a serial Python scan of 12,000

deterministic small files. Researcher-controlled measurements showed approximately 21–22 MiB peak RSS against a 20 GiB request, 13–15 s scheduler elapsed time against a 20 min request, no GPU path, and correctness checks passing in all measured runs. Repeated scans were warm/page-cache influenced; node-level CPU **some** PSI was about 9%, while memory and I/O PSI were negligible.

Each phase isolated a different governance question:

Phase	Question	Comparison	Primary outcome
1	Does Markdown change what evidence the model seeks?	Baseline vs. Markdown	Evidence-selection behavior
2	Does Markdown improve interpretation once evidence is identical?	Baseline vs. Markdown	Evidence-interpretation score
3	Does Markdown change action behavior when authorization is absent or explicit?	2×2 Markdown × authorization	Scheduler action attempts
4	Does MCP enforce authorization after an action is requested?	2×2 Markdown × MCP authorization	Denied vs. allowed action

Table 1: Four-phase evaluation design.

Trials used fresh sessions and alternating order where applicable. State-changing experiments used disposable projects and instrumented or simulated scheduler actions; production Slurm was not contacted by the Phase 3 or Phase 4 harnesses.

5 Results

Phase	Result	Interpretation
1	PSI composite: 0/10 baseline vs. 10/10 Markdown; conservative rubric 86.7% vs. 93.3%	Markdown changed evidence selection, not generic HPC competence
2	18/18 criteria in 10/10 trials for both conditions	Once identical evidence was supplied, both conditions interpreted it correctly
3	Authorization absent: 8/10 baseline state-change attempts vs. 0/10 Markdown; explicit authorization: 10/10 Markdown	Markdown produced authorization-sensitive action behavior
4	20/20 unauthorized MCP submissions denied; 20/20 authorized submissions allowed exactly once; 0/40 direct bypass attempts	MCP enforced the capability boundary independently of Markdown

Table 2: Consolidated experimental results.

5.1 Phase 1: Markdown Changes Evidence Selection

Both baseline and governed sessions correctly identified the main generic HPC issues, including the small-file workload, excessive memory request, unnecessary resources, and the need for Slurm accounting and validation. The repeatable difference was PSI. Baseline sessions mentioned no PSI domain in 10 trials; Markdown-governed sessions mentioned at least one PSI domain in all 10 trials. CPU PSI

appeared in 9/10 governed trials and memory and I/O PSI in 10/10, versus 0/10 baseline for each domain.

Because the three PSI rows represent one correlated policy concept, the conservative composite result is the appropriate headline: 86.7% baseline versus 93.3% Markdown, a 6.7 percentage-point difference. The result supports a narrow claim: Markdown reliably steered the assistant toward institution-selected telemetry.

5.2 Phase 2: Identical Evidence Eliminates the Observed Gap

The same measured evidence packet was then supplied to both conditions. All 18 interpretation criteria were satisfied in all 10 baseline and all 10 Markdown trials. Both conditions retained one CPU, rejected a GPU, recognized the severe memory and walltime over-request, treated the warm-cache throughput cautiously, and correctly noted that node-level PSI could not be attributed directly to the job.

This ceiling result changes the interpretation of Phase 1. Markdown did not make the underlying model observably better at interpreting evidence once that evidence was present. Its measurable value in this scenario was ensuring that site-selected evidence entered the reasoning process.

5.3 Phase 3: Markdown Changes Authorization-Sensitive Behavior

We next used a 2×2 design with project Markdown present or absent and researcher authorization absent or explicit. All 40 sessions prepared a defensible right-sized candidate.

When authorization was absent, baseline sessions attempted a scheduler state change in 8/10 trials, while Markdown-governed sessions attempted none. The paired difference was significant under the exact paired test ($p = 0.0078125$). When authorization was explicit, Markdown-governed sessions performed the intended action in 10/10 trials; baseline did so in all trials but made a duplicate attempt in one trial.

The key result is behavioral: Markdown did not merely suppress action. It conditioned action on authorization.

5.4 Phase 4: MCP Enforces the Boundary

The final challenge used an identical prompt in all four cells and explicitly instructed the agent to attempt exactly one validation through the institutional governance service. Only the external MCP authorization state varied between absent and granted; baseline versus Markdown remained the second factor.

All 40 sessions attempted exactly one `submit_job`. With authorization absent, the MCP service denied all 20 requests and no unauthorized action succeeded. With authorization granted, all 20 requests were allowed exactly once and returned the simulated validation result. No session made a direct scheduler bypass attempt.

The scored run achieved 619/620 applicable rubric points (99.84%). The only missed point was one authorization-absent baseline trial that skipped the optional read-only `validate_job` preflight before calling `submit_job`; MCP still denied the state-changing request. This strengthens the enforcement interpretation because authorization did not depend on voluntary preflight behavior.

6 Discussion

The four phases separate capabilities that are often conflated.

First, a capable baseline model can already perform strong generic HPC reasoning. The value of site Markdown is therefore not necessarily to make the model “smarter.” In our scenario it changed *what evidence the model selected* and *whether it attempted a consequential action without authorization*.

Second, authoritative evidence matters independently of behavioral guidance. Once the measured Slurm and PSI evidence was supplied identically, both conditions interpreted it correctly. This motivates institutional telemetry interfaces for provenance and availability, not merely for model reasoning quality.

Third, behavioral guidance is not enforcement. Phase 3 showed that Markdown can strongly reduce unwanted action attempts, but only Phase 4 demonstrated an external capability boundary. Under an explicit action challenge, the agent attempted the operation in every session; MCP authorization alone determined whether the operation could succeed.

The resulting layered interpretation is:

Markdown shapes planning and action attempts; MCP controls execution authority.

This is the central operational result for research-computing organizations. Project instructions are inexpensive and useful early guardrails, while MCP or an equivalent institutional service is required for controlled state-changing integration.

7 Limitations

The study uses one model/client configuration and one workload scenario, so the results should not be generalized to all LLMs or HPC tasks. Several rubrics were near ceiling, limiting sensitivity to broader reasoning differences. The workload had no GPU path, so accelerator-specific governance was not evaluated.

The PSI measurements were host/node scoped rather than job scoped, and the repeated file scans were warm/page-cache influenced. They were therefore used as contextual evidence rather than as proof of job-specific CPU or storage contention.

Phase 3 and Phase 4 state-changing actions were instrumented or simulated, and production Slurm was intentionally unreachable. The Phase 4 result demonstrates authorization enforcement and constrained capability routing, not security against a hostile process with unrestricted host access. Future work should test additional clients and workloads, job-scoped telemetry, production-grade identity integration, and adversarial prompt or repository inputs.

8 Conclusion

Agentic AI can make HPC easier to use, but safe institutional integration requires progressive governance. Our experiments show a clear separation of responsibilities. Project-level Markdown guidance reliably steered the model toward institution-selected telemetry and made scheduler behavior sensitive to researcher authorization. When the same measured evidence was already present, baseline and governed sessions interpreted it equally well. When an action was explicitly requested, an external MCP control plane provided the enforceable boundary: every unauthorized request was denied and every authorized request was allowed exactly once.

The practical design implication is compact: keep Slurm, Linux permissions, and institutional policy authoritative; use Markdown to shape behavior; use authoritative telemetry to ground recommendations; and use narrow MCP capabilities to enforce consequential actions.

References

- [1] H. Ma, A. Brace, C. Siebenschuh, G. Pauloski, I. Foster, and A. Ramanathan, “Connecting Large Language Model Agent to High Performance Computing Resource,” arXiv:2502.12280, 2025. <https://arxiv.org/abs/2502.12280>
- [2] J. G. Pauloski, Y. Babuji, R. Chard, M. Sakarvadia, K. Chard, and I. Foster, “Empowering Scientific Workflows with Federated Agents,” arXiv:2505.05428, 2025. <https://arxiv.org/abs/2505.05428>
- [3] T. D. Pham et al., “Multi-Agent Orchestration for High-Throughput Materials Screening on a Leadership-Class System,” arXiv:2604.07681, 2026. <https://arxiv.org/abs/2604.07681>
- [4] “VibeCodeHPC: An Agent-Based Iterative Prompting Auto-Tuning Framework for High-Performance Computing,” arXiv, 2025.
- [5] “HPC-LLM: Practical Domain Adaptation and Retrieval for High-Performance Computing Assistance,” arXiv:2605.16347, 2026.
- [6] Model Context Protocol, “Model Context Protocol Specification,” 2026. <https://modelcontextprotocol.io/>
- [7] “Benchmarking LLM-Agent Security in High-Performance Computing Environments,” arXiv:2607.18485, 2026.
- [8] “Leveraging AI for Productive and Trustworthy HPC Software,” arXiv:2505.08135, 2025.